

Absolutely 😊🔥 We'll pick up **exactly here** and won't loop back over invalidation.

Lesson 6 — Modern Next.js Caching: Cache Components

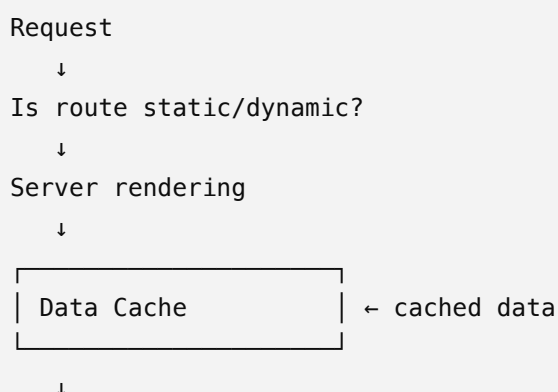


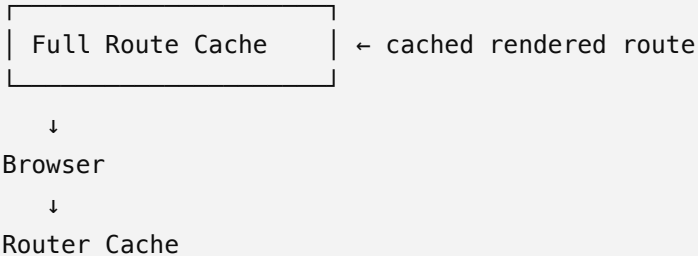
This is the next layer of the mental model.

Before we touch APIs, I want you to understand **why Cache Components exist**, because otherwise `use cache`, `cacheLife()`, and `cacheTag()` can feel like random new APIs.

1. First: the older mental model

The model we just learned was roughly:





And we had concepts like:

```
fetch(url, {
  next: { revalidate: 60 }
})
```

or:

```
fetch(url, {
  cache: "no-store"
})
```

which influenced how Next.js cached the data and consequently whether the route could be cached.

2. The problem with that model

Imagine this page:

/product/123



Product information	← changes occasionally
Product reviews	← changes frequently
Current user's cart	← personalized
Recommended products	← expensive computation

Suppose:

Product information

Can be cached:

Product #123



Recommendations

Can also be cached:

Recommendations for Product #123



Cart

Cannot simply be shared because:

User A → Cart A

User B → Cart B



So the page contains a mixture of:

CACHEABLE

+

DYNAMIC



The older mental model often makes you think in terms of:

"Is this entire route static or dynamic?"

But modern Next.js wants you to think more granularly:

Which parts of the UI can be cached, and which parts must remain dynamic?



That's the key idea.

3. Enter Cache Components

Modern Next.js introduces:

```
"use cache"
```

This allows you to explicitly say:

"Cache the result of this component/function."

For example:

```
async function getProduct(id: string) {  
  "use cache"  
  
  const product = await db.product.findUnique({  
    where: { id }  
  })  
  
  return product  
}
```

Conceptually:

getProduct(123)

↓

"use cache"

↓

Cached result
Product #123



Instead of thinking:

Entire route = cached



we can think:

Route

|

├ Product → CACHE

|

├ Recommendations → CACHE

|

└ User Cart → DYNAMIC



That is a **much more granular model**.

4. Very important: "use cache" is NOT "use client"

This is an easy interview trap. 😊

You've already learned:

```
"use client"
```

means:

This module/component needs to participate in Client Component behavior.

Whereas:

```
"use cache"
```

means:

Cache the result of this server-side computation/rendering according to the Cache Components model.

So:

```
"use client"
```

↓

Client Component boundary

```
"use cache"
```

↓

Cache boundary



They solve completely different problems.

5. Now `cacheLife()`

Suppose we say:

```
"use cache"
```

```
cacheLife("hours")
```

Conceptually we're saying:

Cache this result, and use this caching lifetime.

For example:

```
async function getProducts() {  
  "use cache"  
  
  cacheLife("hours")  
  
  return db.product.findMany()  
}
```

Mental model:

getProducts()

↓

"use cache"

↓

cacheLife("hours")

↓

Cached computation

lifetime = hours

So:

"use cache"

answers:

Should this computation be cacheable?

cacheLife()

answers:

How long / under what lifetime should this cached result live?

6. Then `cacheTag()`

Now imagine:

```
async function getProduct(id: string) {  
  "use cache"  
  
  cacheTag(`product:${id}`)  
}
```

```
return db.product.findUnique({
  where: { id }
})
}
```

We attach a tag:

Product #123



cache key



product:123

Now later, if Product 123 changes, we have a way to identify the cached data associated with it.

Conceptually:

cacheTag("product:123")



Cached Product #123



invalidation

So:

"use cache"



make it cacheable

cacheLife()



control lifetime

cacheTag()



give it an invalidation identity

That's the first mental model I want you to lock in.

7. Now comes the really interesting part: PPR

PPR = Partial Prerendering

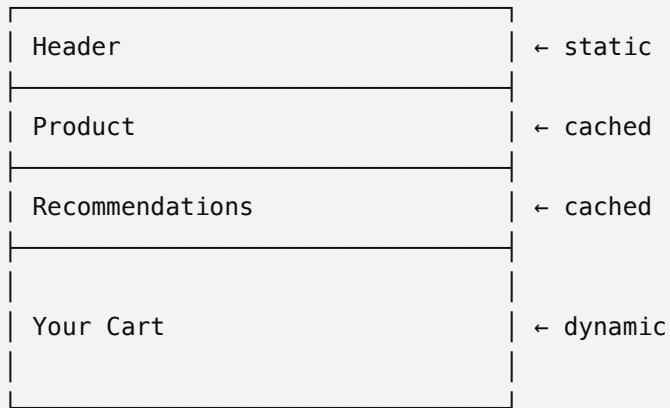
This is where Cache Components becomes much more powerful.

Imagine:

/product/123



has:



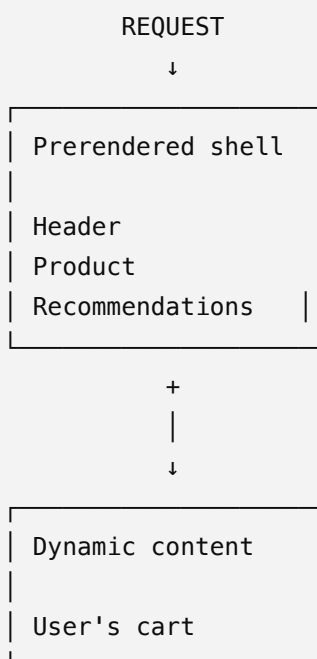
Previously you might think:

Dynamic cart
↓
Entire route becomes dynamic



The modern model can instead preserve the cacheable/prerenderable parts while leaving a dynamic portion to resolve separately.

Conceptually:



That's the basic intuition behind **Partial Prerendering**.

8. The mental shift 🔥

Old question:

"Is this route static or dynamic?"

Modern question:

"Which parts can be prerendered/cached, and which parts need to remain dynamic?"

That's a *much* more useful way to reason about modern Next.js.

9. One example

Consider:

```
export default async function ProductPage() {  
  return (  
    <>  
      <Header />  
  
      <Product />  
  
      <Recommendations />  
  
      <Cart />  
    </>  
  )  
}
```

Imagine:

```
Header  
  ↓  
static  
  
Product  
  ↓  
"use cache"  
  
Recommendations  
  ↓
```



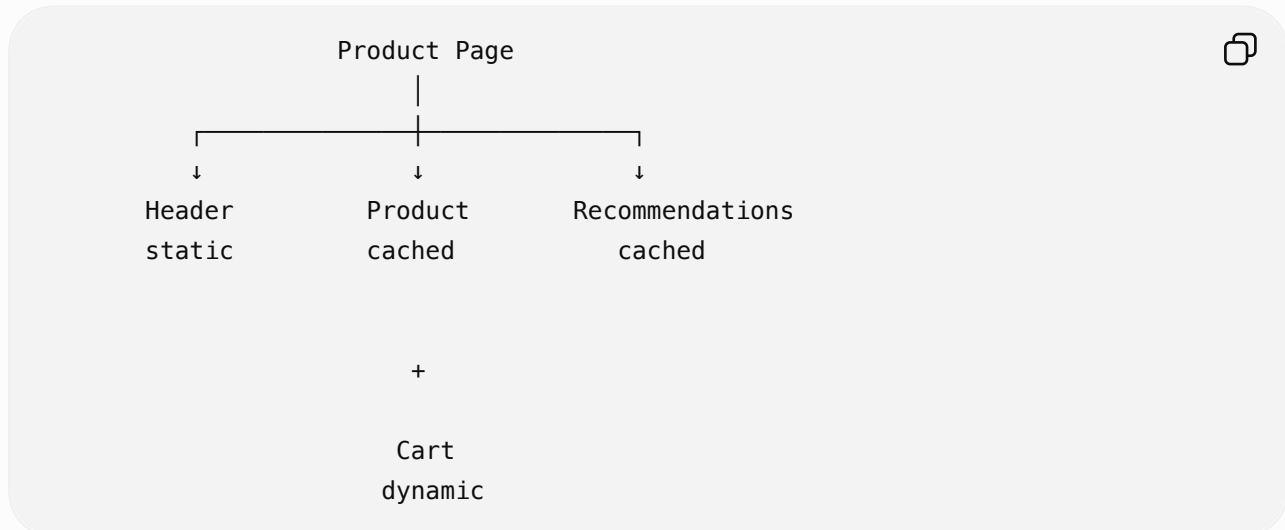
"use cache"

Cart



dynamic

Then conceptually:



So one page can contain different caching/rendering characteristics at different boundaries.

That's the important conceptual upgrade.

10. And here's the interview-level distinction

If an interviewer asks:

"What does `use cache` do?"

Don't say:

"It caches the page."

✗ Too broad.

Say:

"`use cache` marks a server-side computation or component for caching under Next.js Cache Components. It allows caching at a finer granularity instead of treating the entire route as one cacheable unit."

That's a much stronger answer.

"What does `cacheLife()` do?"

"It configures the lifetime/profile of a cached result."

"What does `cacheTag()` do?"

"It associates a tag with cached content so that related cached entries can be invalidated or revalidated by tag."

"What is PPR?"

"Partial Prerendering allows a route to have a prerendered shell while dynamic portions are resolved separately, so dynamic content doesn't necessarily prevent the rest of the UI from being prerendered."



That's the foundation.

11. Our new mental model

I want you to keep **both models**, because you'll encounter both in real projects/interviews:

Earlier model

```
Route
├── Data Cache
├── Full Route Cache
└── Router Cache
```



Modern Cache Components model

```
Route
├── cached component/computation
│   ├── use cache
│   ├── cacheLife
│   └── cacheTag
├── cached component
└── dynamic component
```



↓
PPR

And underneath, the request can still involve things like:

```
Browser
  ↓
CDN / HTTP cache
  ↓
Next.js server / deployment
  ↓
Cache Components
  ↓
dynamic work
  ↓
response
```



So **Cache Components** doesn't mean "throw away everything we learned."

It's a newer way of expressing **what should be cached/prerendered and at what granularity.**

🎯 One thing I want you to notice

We are **not yet** going deep into the old Data Cache → Full Route Cache mechanics again.

We've already done that.

From here our sequence should be:

```
Lesson 6.1
"use cache"
  ↓
what exactly gets cached?
  ↓
cache key / arguments
  ↓
cacheLife()
  ↓
cacheTag()
  ↓
dynamic data inside cached components
  ↓
PPR
  ↓
full request lifecycle
```



↓
interview questions

And the next piece is `use cache` under the hood — especially what becomes the cache key and why function arguments matter. That's where this gets really interesting. 😎🔥



ok

Perfect 😊🔥 Let's continue with Lesson 6.1 — `use cache` under the hood.

6.1 — What exactly happens with `"use cache"`?

Start with this:

⌵ TypeScript

```
async function getProduct(id: string) {  
  "use cache"  
  
  return db.product.findUnique({  
    where: { id }  
  })  
}
```



The most important question is:

If `getProduct()` is called multiple times, how does Next.js know whether the cached result belongs to Product 123 or Product 456?

That brings us to the **cache key**.

1. Think of `"use cache"` as creating a cached function

Without caching:

```
getProduct(123)  
  ↓  
database  
  ↓  
Product 123
```



Every call can execute the computation again.

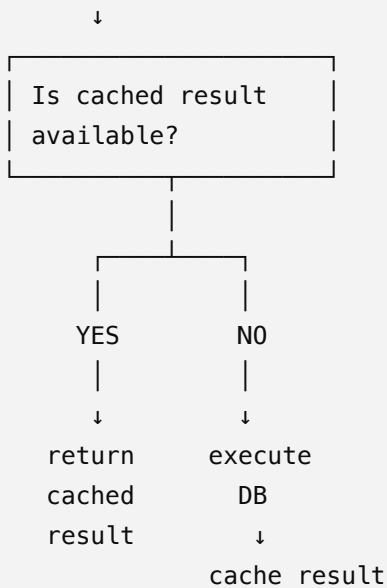
With:

```
</> TypeScript
```

```
"use cache"
```

we conceptually get:

```
getProduct(123)
```



2. Function arguments matter

Suppose:

```
</> TypeScript
```

```
async function getProduct(id: string) {  
  "use cache"  
  
  return db.product.findUnique({  
    where: { id }  
  })  
}
```

We call:

```
</> TypeScript
```

```
getProduct("123")
```

and:

```
</> TypeScript  
getProduct("456")
```



These should obviously produce different cached results.

Conceptually:

```
getProduct("123")  
  ↓  
cache entry A  
  ↓  
Product 123  
  
getProduct("456")  
  ↓  
cache entry B  
  ↓  
Product 456
```



So the arguments participate in identifying the cached computation.

Think:

```
Function  
+  
Arguments  
  ↓  
Cache identity
```



Not literally as a simplistic string concatenation, but that's the mental model you want.

3. Why this is powerful

Imagine:

```
</> TypeScript  
  
async function getProduct(id: string) {  
  "use cache"  
  
  return db.product.findUnique({  
    where: { id }  
  })  
}
```



```
} )  
}
```

Then:

```
User A → /products/123  
User B → /products/123  
User C → /products/123
```

All three need:

```
Product 123
```

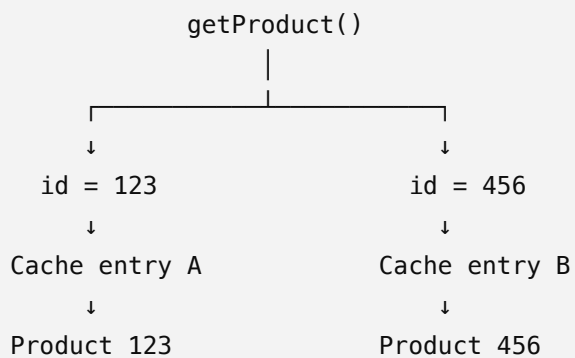
The cached computation can potentially be reused.

But:

```
User A → /products/123  
User B → /products/456
```

need different cached results.

So:



4. This is different from user-specific data

Now here's where things get interesting. 🤖

Suppose we write:

```
</> TypeScript  
  
async function getCart() {  
  "use cache"
```

```
const user = await getCurrentUser()

return getCartForUser(user.id)
}
```

We have a problem conceptually:

User A → Cart A
User B → Cart B



You **cannot accidentally create one globally shared cached result:**

```
getCart()
  ↓
"Cart A"
```



and then give that to User B. 😬

This is why understanding **what data is inside a cacheable computation and what inputs/context it depends on** is critical.

5. Dynamic APIs are important here

You've already learned these:

```
cookies()
headers()
searchParams
```



They represent request-dependent information.

For example:

```
</> TypeScript

const theme = cookies().get("theme")
```



depends on the current request/user.

Compare:

```
getProducts()
  ↓
same for everyone
  ↓
```



excellent caching candidate

getUserCart()

↓

depends on user

↓

dynamic/personalized

This is one of the major reasons Cache Components introduces explicit cache boundaries.

6. Cache boundary mental model

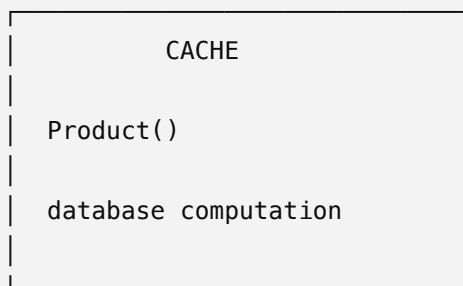
Imagine:

</> TypeScript

```
async function Product() {  
  "use cache"  
  
  const product = await getProduct()  
  
  return <div>{product.name}</div>  
}
```



We can visualize:



Then:

</> TypeScript

```
async function Cart() {  
  // dynamic  
  const cart = await getCart()  
  
  return <CartUI cart={cart} />  
}
```



becomes:



And that's exactly the granular architecture we're learning.

7. One subtle but VERY important point

Don't think:

"use cache" = browser cache

✗ Nope.

Don't think:

"use cache" = CDN cache

✗ Nope.

And don't think:

"use cache" = Router Cache

✗ Nope.

It's a **server-side Next.js Cache Components mechanism**.

So keep these concepts separate:

Browser

↓

Browser Cache

CDN



HTTP/CDN Cache

Next.js



Cache Components

Browser navigation



Router Cache

Different layers. Different purposes.

8. Where `cacheLife()` enters

Once you've marked something:

```
</> TypeScript
```



```
"use cache"
```

you can configure its lifetime:

```
</> TypeScript
```



```
cacheLife("hours")
```

So:

```
"use cache"
```



WHAT can be cached?

```
cacheLife()
```



HOW LONG / WHAT lifetime profile?

```
cacheTag()
```



HOW do we identify related cached data?



This three-part mental model is 🔑 .

🧠 Interview checkpoint

If I ask you:

"What happens when you put `use cache` inside a function that takes `id` ?"

A good answer would be:

"`use cache` makes the function's server-side result cacheable. The function's inputs, such as `id` , contribute to the identity of the cached computation, so calls for different IDs can have separate cached results."

That's the level we're aiming for.

🎯 Next: 6.2 — `cacheLife()`

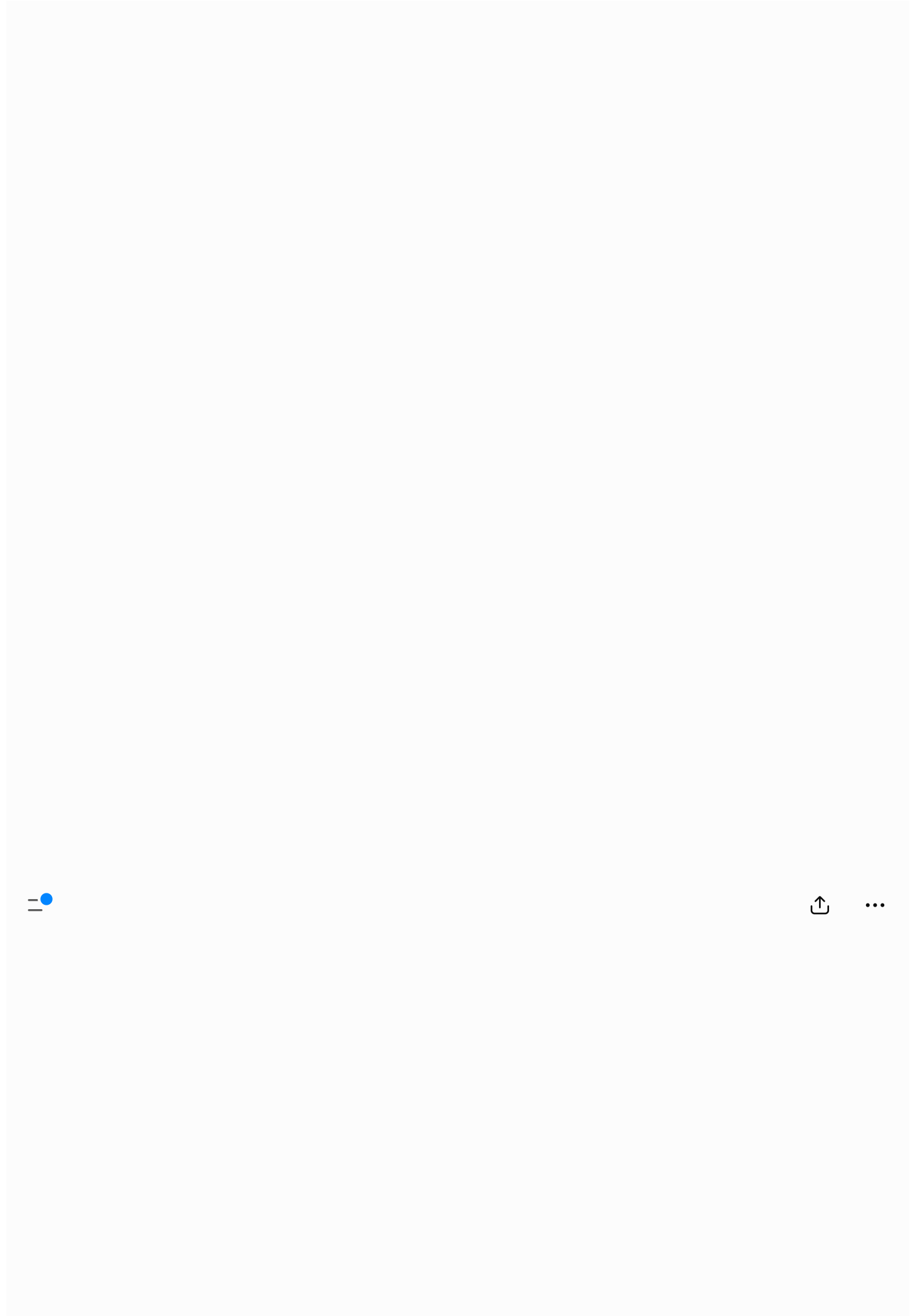
We'll go deeper into:

```
cacheLife()  
  ↓  
stale  
  ↓  
revalidate  
  ↓  
expire  
  ↓  
what happens on a request  
  ↓  
fresh vs stale vs expired
```



And **this is where it connects beautifully with the TTL/revalidation concepts you already learned.** 🔥





Yes — this is the **exact confusion to clear up**. 😊 Once you understand this, Server Components become much less magical.

1. "If Server Component code isn't in the browser, how does it show on screen?"

Because **the browser doesn't need the Server Component code to display its result.**

Think about a restaurant 🍕 :

```
Chef (server)
  ↓
uses recipe + ingredients
  ↓
prepares pizza
  ↓
waiter delivers pizza
  ↓
Customer sees pizza
```



The customer doesn't need the **recipe** or the **kitchen** to see the pizza.

Same idea:

```
SERVER
Server Component code
  ↓
executes
  ↓
produces result
  ↓
HTML / RSC payload
  ↓
BROWSER
  ↓
screen
```



2. Concrete example

You write:

```
</> TypeScript

// app/products/page.tsx

export default async function ProductsPage() {
  const products = await db.product.findMany();

  return (
    <main>
      <h1>Products</h1>

      {products.map(product => (
        <div key={product.id}>
          {product.name}
        </div>
      ))}
    </main>
  );
}
```



```
    </div>
  )})
</main>
);
}
```

This is a **Server Component**.

The browser does NOT need:

- ✗ ProductsPage.tsx
- ✗ db.product.findMany()
- ✗ database credentials
- ✗ database connection



The server executes:

```
ProductsPage()
  ↓
DB query
  ↓
React renders result
```



Suppose the result is:

```
Products:
iPhone
MacBook
AirPods
```



The browser can receive/render markup representing that result:

```
HTML

<main>
  <h1>Products</h1>
  <div>iPhone</div>
  <div>MacBook</div>
  <div>AirPods</div>
</main>
```



So the browser says:

"Cool, I have markup. I'll paint this on the screen."

It doesn't need to know **how the server produced it**.

3. Then what is RSC doing?

Here's the important distinction:

Server Component CODE

↓

runs on server

↓

Server Component RESULT

↓

RSC payload

↓

browser React



The browser receives the **result/representation**, not the original server implementation.

That's why:



Browser:

"Let me execute ProductsPage.tsx"



vs



Browser:

"I received the result of ProductsPage."

4. Can you SEE Router Cache in your browser? 🙄

Yes — but there's an important caveat.

You won't normally open DevTools and see:

Router Cache

/products/123

/products/456



as a nice readable cache panel.

Next.js doesn't expose Router Cache as a simple Chrome DevTools storage bucket like:

Application

├ Local Storage

├ Session Storage



```
| IndexedDB  
| Router Cache ← ❌ not normally here
```

Router Cache is an **internal client-side cache maintained by the Next.js App Router**, not ordinary `localStorage`.

5. But you CAN inspect evidence of it 🔍

This is actually a great experiment for you.

Open your Next.js app in Chrome.

Then:

```
Chrome DevTools  
↓  
Network
```



Navigate using:

```
</> TypeScript  
  
<Link href="/products/123">
```



rather than:

```
HTML  
  
<a href="/products/123">
```



You can inspect the network requests and see requests associated with the App Router/RSC navigation.

Depending on your Next.js version and dev/prod mode, you'll see RSC-related request/response information.

You may encounter headers such as:

```
RSC  
Next-Router-State-Tree  
Next-Router-Prefetch
```



and responses associated with the RSC protocol.

The exact presentation varies between development/production and Next.js versions.

6. Here's a really good experiment

Make two pages:

```
/products/123  
/products/456
```



and navigate:

```
123  
↓  
456  
↓  
123
```



using:

```
</> TypeScript  
  
<Link href="/products/123">
```



Watch:

```
DevTools  
→ Network
```



You'll be able to observe the App Router making RSC/navigation requests.

Then try:

```
/products/123
```



→

```
/products/456
```



→

```
/products/123
```



The Router Cache can allow previously received route information to be reused according to its current caching behavior.

7. Can you visually see the cached Server Component?

This needs a careful distinction.

✗ You generally won't see:

Next.js Server Cache



Product component
cached result

inside Chrome DevTools.

Why?

Because that cache is **on the server**, not in your browser.

Remember:

"use cache"



SERVER-SIDE CACHE

while:

Router Cache



CLIENT-SIDE

They're different.

8. So where can you inspect each one?

Thing	Where it lives	Can you inspect it?
Server Component source	Server/build	Not as browser JS
"use cache" result	Server-side cache	Not directly through normal browser DevTools
RSC response	Network	✔ Yes
HTML	Network → document response	✔ Yes

Thing	Where it lives	Can you inspect it?
Client JS	Network/Sources	✅ Yes
Router Cache	Next.js client runtime	⚠️ Not as a normal DevTools storage panel
localStorage	Browser	✅ Yes
sessionStorage	Browser	✅ Yes
HTTP/browser cache	Browser DevTools	✅ Partially inspectable

9. And here's a SUPER useful experiment for you

Open:

Chrome DevTools
→ Network
→ click the document request
→ Response



You'll be able to inspect the initial HTML response.

Then navigate with `<Link>` and inspect the subsequent requests.

You'll start seeing the difference between:

INITIAL LOAD

HTML
+
RSC information
+
Client JS



and:

CLIENT NAVIGATION

RSC-related response
+
existing browser document



That distinction is really important for App Router interviews.

10. One more thing: don't confuse "showing" with "executing"

This is the key sentence:

A browser can display the output of code without possessing or executing the code that generated that output.

For example:

Server Component:

```
<h1>{product.name}</h1>
  ↓
server knows product.name = "iPhone"
  ↓
result
  ↓
<h1>iPhone</h1>
  ↓
browser displays it
```

The browser doesn't need:

`</>` TypeScript

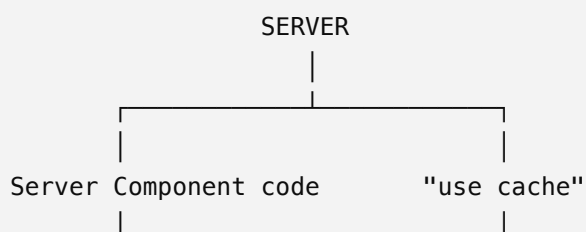
```
<h1>{product.name}</h1>
```

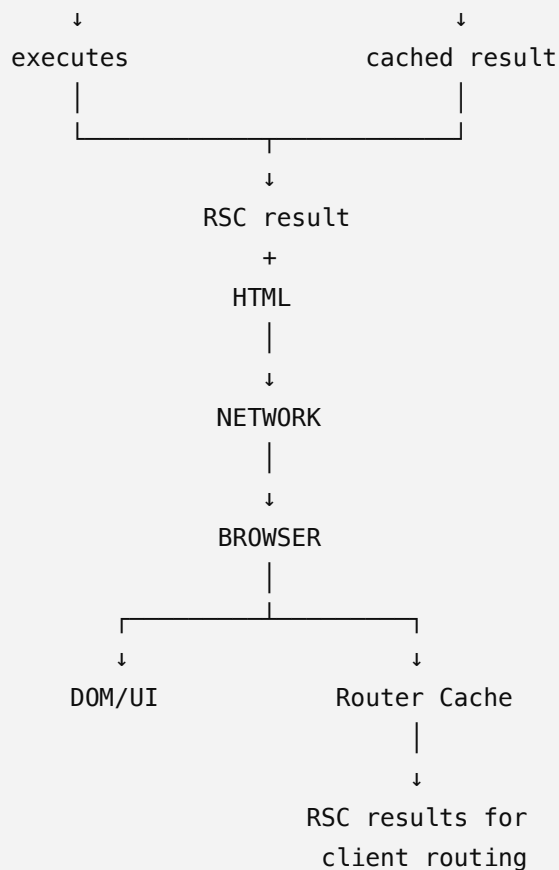
to be able to display:

iPhone

Final picture

Keep these two worlds separate:





🔥 The one thing to remember:

Server Component code stays on the server. Its *result* crosses the network.

And yes, **you can absolutely open DevTools and inspect the HTML and RSC network traffic.** You just generally can't open a little "Router Cache" folder and browse it like localStorage.

If you want, we can next do a **hands-on DevTools experiment** where I give you a tiny Next.js page and tell you **exactly what to click in Chrome DevTools and what you should expect to see.** That'll make this stick for real. 🙄



i willdo it after i build nextjs app ..next

Absolutely 🤪 We'll do the DevTools experiment **after you build the Next.js app** so you can actually see the HTML, RSC requests, navigation, and caching instead of just imagining them.

Next → Lesson 6.8 🚀

We'll move into:

Hydration vs Reconciliation vs RSC Updates

We'll answer:

Server Component renders on server

↓

HTML reaches browser

↓

Client Component JS arrives

↓

What exactly gets hydrated?

↓

What does React reconcile?

↓

What happens during router.refresh()?

↓

What happens during <Link> navigation?

And we'll use one small example throughout so the differences are crystal clear. 🧠



how any more?

Haha 🤔 you're asking **how many lessons are left in the whole Next.js roadmap**, right?

You completed **Lesson 4**, and we're currently around **Lesson 6.7** in the deeper caching/RSC section.

 Remaining roadmap

Lesson	Topic	Status
6.8	Hydration vs Reconciliation vs RSC Updates	➡️ SOON Next
6.9	Full request lifecycle — putting everything together	🕒
7	Next.js Performance	🕒
8	Authentication & Authorization	🕒

Lesson	Topic	Status
9	Middleware / Proxy & request interception	🕒
10	Route Handlers & APIs	🕒
11	Error Handling & Loading UX	🕒
12	Forms, Server Actions & Mutations — interview deep dive	🕒
13	SEO, Metadata & OG Images	🕒
14	Images, Fonts & Scripts	🕒
15	Deployment, Vercel, CDN & Edge	🕒
16	Testing Next.js apps	🕒
17	Security & production best practices	🕒
18	Next.js System Design	🕒
19	Interview questions + debugging scenarios	🕒
20	Build the Web Performance Analyzer 🚀	🕒

So roughly 14 major lessons/sections remain.

But don't worry — several of those are much smaller than the caching/RSC section we've just gone through. 😂

And 6.8 → 6.9 is basically the final deep architectural piece. After that we'll move into practical/interview-focused Next.js topics.

Your journey so far

- Lesson 1 → Server vs Client Components

Lesson 2 → App Router & Routing

Lesson 3 → Navigation & Routing details

Lesson 4 → Data Fetching

Lesson 5 → Mutations & Server Actions

Lesson 6 → Caching + Rendering architecture
 - └ Cache Components
 - └ use cache
 - └ cacheLife
- ✓

✓

✓

✓

✓

🔄

✓

✓

✓



└─ cacheTag	✓
└─ PPR	✓
└─ RSC	✓
└─ 6.8 Hydration/Reconciliation	→ SOON

So we're **well past the fundamentals now.** 🔥

And after the roadmap, we'll use all of this in your **Web Performance Analyzer project**, which is where the concepts should really become second nature.



ChatGPT can make mistakes. Check important info.